## 🗺️ MASTER UPSKILLING ROADMAP — FULL CHECKLIST

**How to use this:** Go at your own pace. Each domain has subtopics as checkboxes, practice sources, and a capstone project. Do basics + LLM track in parallel. Don't rush — depth > speed.

---

### ⚙️ DOMAIN 0 — Python Foundations (Polish, Don't Start Over)

You already know Python. This is just filling gaps. Do this fast — 1-2 weeks max.

**NumPy**

- Arrays, indexing, slicing, reshaping
- Broadcasting rules
- Vectorized operations (no loops)
- Linear algebra ops — dot product, matrix multiply, transpose
- Random number generation, seeds

**Pandas**

- Series vs DataFrame, dtypes
- loc, iloc, conditional filtering
- groupby, agg, pivot_table
- merge, join, concat
- Handling missing values, duplicates
- apply, map, lambda functions
- read_csv, read_excel, read_json, write to CSV

**Practice Sites:**

- **W3Schools NumPy/Pandas** — quick reference
- **Kaggle Learn: Pandas** — free, hands-on, gives you real dataset exercises
- **LeetCode (Easy SQL)** — do 10 easy SQL problems alongside this

### 📦 Capstone Project 0:

**"Exploratory Sales Dashboard"** — Take any Kaggle CSV dataset (e.g., Superstore Sales), do full cleaning with Pandas, compute KPIs using groupby and agg, export a summary table. Push to GitHub. 🔗 Dataset source: **kaggle.com/datasets** → search "superstore sales"

---

## 🐍 DOMAIN -1 : Python for DS/AI (Add This Before Domain 0)

Core Python — must be second nature

- Data types — lists, dicts, sets, tuples and when to use each
- List comprehensions and dict comprehensions — write clean one-liners
- Functions — args, kwargs, default values, return multiple values
- Lambda functions — used everywhere in Pandas and ML pipelines
- Error handling — try, except, finally — critical for production agents
- File handling — read/write JSON, CSV, TXT — daily use in DS
- Iterators, generators — for handling large data efficiently
- Decorators — basic understanding, you'll see them in FastAPI everywhere
- *args and **kwargs — understand deeply, LangChain uses these constantly

OOP — Medium depth, not deep

- Classes, objects, __init__
- Inheritance — understand it, you'll read it more than write it
- Dunder methods — __str__, __len__, __call__
- Why sklearn, PyTorch, LangChain are all OOP under the hood

Python for Production — this is what separates juniors from seniors

- Virtual environments — venv, conda
- requirements.txt, environment.yml
- .env files — storing API keys safely, python-dotenv
- Logging — proper logging instead of print statements
- Type hints — def func(name: str) -> int — used in all serious codebases
- Async Python — async, await, asyncio — critical for LangChain and FastAPI
- Pydantic — data validation, used heavily in FastAPI and LangChain output parsers

Test for Python Level 2: Can you write a clean Python class with proper type hints, logging, env variable loading, and async methods without Googling? That's your bar.

Sources:

- "Fluent Python" by Luciano Ramalho — not full book, read chapters on data structures, functions, OOP, async only. Best Python book ever written.
- Real Python (realpython.com) — every topic above has a clean tutorial here, free
- Python docs — for async specifically, the official docs are actually the clearest
-

📊 **DOMAIN 1 — Statistics & Probability (Foundation of Everything)**

Most DS candidates are weak here. This is what separates you in senior interviews.

**Descriptive Statistics**

- Mean, median, mode, range
- Variance, standard deviation, IQR
- Skewness, kurtosis
- Percentiles, quartiles, boxplots

**Probability**

- Basic probability rules (AND, OR, conditional)
- Bayes' theorem — understand it deeply, not just the formula
- Probability distributions — Normal, Binomial, Poisson, Uniform
- Central Limit Theorem — why it matters in ML

**Inferential Statistics**

- Hypothesis testing — null vs alternate hypothesis
- p-value, significance level, Type I & II errors
- t-test, chi-square test, ANOVA
- Confidence intervals
- A/B testing — how it actually works end to end

**Correlation & Relationships**

- Pearson vs Spearman correlation
- Covariance
- Multicollinearity — what it is and why it kills models

**Practice Sites:**

- **StatQuest on GitHub + notes** — best stats explanations anywhere
- **Khan Academy Statistics** — if any concept is unclear, go here
- **Seeing Theory** (seeing-theory.brown.edu) — visual probability, bookmark this
- **Mode Analytics SQL Tutorial** — for stats + SQL combo

📦 **Capstone Project 1:**

**"A/B Test Analyzer"** — Take a mock A/B test dataset, compute conversion rates, run a t-test, calculate statistical significance, visualize the result, write a 1-page conclusion like a real analyst would. Push to GitHub. 🔗 Dataset: Kaggle → "A/B testing dataset" or generate synthetic data with NumPy

---

📉 **DOMAIN 2 — Data Analysis & EDA**

This is your daily bread at work. Master this completely.

**EDA Techniques**

- Shape, dtypes, info, describe
- Missing value analysis — heatmaps, percentage missing
- Univariate analysis — histograms, KDE plots, boxplots
- Bivariate analysis — scatter plots, correlation heatmaps, pairplots
- Categorical analysis — value_counts, bar charts, countplots
- Outlier detection — IQR method, Z-score method

**Visualization**

- Matplotlib — figure, axes, subplots
- Seaborn — heatmap, pairplot, violinplot, boxplot, barplot
- Plotly — interactive charts (very impressive in portfolios)
- Storytelling with data — how to present findings, not just plot them

**Feature Engineering Basics**

- Encoding — Label, One-Hot, Target encoding
- Scaling — MinMax, StandardScaler, RobustScaler
- Binning, log transform, polynomial features
- Date/time feature extraction

**Practice Sites:**

- **Kaggle Notebooks** — read top 3 notebooks on any popular dataset, study how they do EDA
- **Towards Data Science (Medium)** — search "EDA case study"
- **DataCamp Projects** — free tier has a few good ones

📦 **Capstone Project 2:**

**"End-to-End EDA Report"** — Pick a messy dataset (Titanic, House Prices, or any domain you like). Do complete EDA, feature engineering, write insights like a business analyst. Use Plotly for interactive charts. Export as a clean Jupyter notebook + push to GitHub. 🔗 Datasets: **kaggle.com/competitions** — Titanic, House Prices are classics

---

🤖 **DOMAIN 3 — Classical Machine Learning**

Must be solid here. Every DS interview will test this. Go deep, not wide.

**Supervised Learning — Regression**

- Linear Regression — OLS, assumptions, residual analysis
- Ridge, Lasso, ElasticNet — regularization, when to use which
- Polynomial Regression
- Evaluation metrics — MAE, MSE, RMSE, $R^2$, Adjusted $R^2$

**Supervised Learning — Classification**

- Logistic Regression — sigmoid, log loss, decision boundary
- Decision Trees — splitting criteria (Gini, Entropy), pruning
- Random Forest — bagging, feature importance
- Gradient Boosting — XGBoost, LightGBM, CatBoost
- SVM — kernels, margin, C parameter
- KNN — distance metrics, choosing K

- Naive Bayes — when it works, when it doesn't
- Evaluation — Accuracy, Precision, Recall, F1, AUC-ROC, Confusion Matrix

**Unsupervised Learning**

- K-Means clustering — inertia, elbow method, silhouette score
- DBSCAN — density-based, good for anomaly detection
- Hierarchical clustering — dendrograms
- PCA — dimensionality reduction, explained variance, when to use

**Model Building Best Practices**

- Train/Validation/Test split — why all three
- Cross-validation — K-Fold, Stratified K-Fold
- Hyperparameter tuning — GridSearchCV, RandomizedSearchCV, Optuna
- Handling class imbalance — SMOTE, class_weight, oversampling
- Feature selection — RFE, SelectKBest, SHAP values
- Model interpretability — SHAP, LIME (this impresses in interviews)
- Pipelines — sklearn Pipeline to avoid data leakage

**Practice Sites:**

- **Kaggle Competitions** — Titanic (classification), House Prices (regression), start here
- **Scikit-learn documentation** — read examples, not just API docs
- **ML from Scratch (GitHub: eriklindernoren)** — implement algorithms from scratch, 1-2 of them, builds deep understanding
- **Interview prep: InterviewQuery.com** — DS interview questions with ML concepts

📦 **Capstone Project 3:**

**"End-to-End ML Pipeline — Churn Prediction"** — Full pipeline: EDA → feature engineering → try 4+ models → compare with evaluation metrics → tune best model → SHAP explainability → save model with joblib → simple prediction script. This is a standard DS portfolio project that impresses recruiters. 🔗 Dataset: Kaggle "Telco Customer Churn" — perfect for this

---

🧠 **DOMAIN 4 — Deep Learning & Neural Networks**

You've done basic CV. Now understand the "why" behind everything.

**Neural Network Fundamentals**

- Perceptron, activation functions — ReLU, Sigmoid, Tanh, Softmax
- Forward pass, backpropagation — understand the math once
- Loss functions — Cross-entropy, MSE, Binary Cross-entropy
- Optimizers — SGD, Adam, RMSprop — when to use which
- Batch size, epochs, learning rate — impact on training
- Overfitting — Dropout, BatchNorm, Early Stopping, L2 reg

**Framework — PyTorch (learn this, not Keras)**

- Tensors, autograd
- Building a model with nn.Module
- DataLoader, Dataset class
- Training loop — forward, loss, backward, step
- Saving and loading models

**Convolutional Neural Networks (CNN)**

- Convolution, pooling, stride, padding — visual understanding
- Classic architectures — VGG, ResNet, EfficientNet (use pretrained)
- Transfer Learning — feature extraction vs fine-tuning
- Data augmentation — torchvision transforms

**Computer Vision — upgrade your current skills**

- OpenCV — image preprocessing, morphological ops, contours
- Object Detection — YOLOv8 (you know this, now go deeper)
- Serving a YOLO model as a REST API with FastAPI
- Video analytics — frame-by-frame processing pipeline

**Transformers & Attention (Bridge to LLMs)**

- Self-attention mechanism — understand visually (Illustrated Transformer by Jay Alammar)
- Encoder-Decoder architecture

- BERT — how it's pretrained, what it's good for
- GPT family — autoregressive generation, how tokens work

**Practice Sites:**

- **fast.ai** — best practical DL course, free, project-first approach
- **PyTorch official tutorials** — pytorch.org/tutorials
- **Papers With Code** (paperswithcode.com) — find benchmarks, implementations
- **Hugging Face** — load pretrained models, fine-tune on small datasets

📦 **Capstone Project 4:**

**"Computer Vision API"** — Train or fine-tune a CNN or YOLO model for a real use case (e.g., defect detection, product classification, face mask detection). Wrap it as a FastAPI endpoint. Document with a Swagger UI. Push to GitHub with a README that shows example API calls. This is a proper portfolio piece. 🔗 Datasets: Roboflow Universe (roboflow.com/universe) — hundreds of labeled CV datasets free

---

🦜 **DOMAIN 5 — LLMs & GenAI (via LangChain Open Tutorial)**

This is your main weapon. Go through the tutorial systematically. Here's the exact checklist mapped to it.

**Block 1 — LangChain Basics** *(Tutorial sections 01-04)*

- LangChain setup, environment, API keys management
- Chains — LCEL (LangChain Expression Language) basics
- Chat models vs LLM models — when to use which
- Prompt templates — ChatPromptTemplate, SystemMessage, HumanMessage
- Output parsers — StrOutputParser, PydanticOutputParser, JsonOutputParser
- Streaming responses
- Model comparison — OpenAI, Anthropic, Groq, local models via Ollama

**Block 2 — Memory & State** *(Tutorial section 05)*

- ConversationBufferMemory
- ConversationBufferWindowMemory
- ConversationSummaryMemory

- Chat history — storing and loading from DB (SQLite, Redis)
- Multi-session, multi-user memory management ← you mentioned this, it's in section 05

**Block 3 — Document Processing** *(Tutorial sections 06-07)*

- Document loaders — PDF, TXT, CSV, Web, YouTube transcript, Notion
- Text splitting strategies — RecursiveCharacterTextSplitter, TokenTextSplitter
- Chunking strategies — chunk size vs overlap — how they affect retrieval
- Semantic chunking — when to use vs fixed chunking

**Block 4 — Embeddings & Vector Stores** *(Tutorial sections 08-09)*

- What embeddings are — vector representation of meaning
- Embedding models — OpenAI, HuggingFace (sentence-transformers), Cohere
- Vector databases — FAISS (local), ChromaDB (local), Pinecone (cloud)
- Indexing strategies — flat vs HNSW
- Similarity search — cosine, dot product, euclidean

**Block 5 — Retrieval & Reranking** *(Tutorial sections 10-11)*

- Basic retriever — similarity search
- MMR (Maximal Marginal Relevance) — diversity in results
- Multi-query retriever — generates multiple queries for better recall
- Contextual compression retriever
- Reranking — Cohere Rerank, BGE Reranker — why it improves RAG
- Ensemble retriever — combining BM25 + vector search (Hybrid Search)

**Block 6 — RAG Pipelines** *(Tutorial section 12)*

- Naive RAG — basic pipeline, understand its limits
- Advanced RAG patterns:
    - HyDE — Hypothetical Document Embeddings
    - Self-RAG — model decides when to retrieve
    - Corrective RAG — validates retrieved docs
    - RAG Fusion — multiple queries merged

- RAG evaluation — RAGAs framework (faithfulness, relevancy, context precision)
- Handling hallucinations in RAG

**Block 7 — Chains & LCEL** *(Tutorial sections 13-14)*

- LCEL pipe syntax — composing chains with |
- Parallel chains — RunnableParallel
- Conditional chains — RunnableBranch
- Custom runnables — RunnableLambda
- Structured output chains — SQL generation, data extraction chains

**Block 8 — Agents & Tools** *(Tutorial section 15)*

- What agents are vs chains
- Tool calling — custom tools, built-in tools (search, Python REPL, file tools)
- ReAct agent pattern — Reasoning + Acting
- Structured output agents
- Error handling and retries in agents

**Block 9 — LangGraph (Production Agents)** *(Tutorial section 17)*

- Graph concept — nodes, edges, state
- StateGraph — defining agent state
- Conditional edges — routing logic
- Supervisor pattern — orchestrator + worker agents
- Multi-agent collaboration graph
- Human-in-the-loop — interrupt and resume
- Memory in LangGraph — short-term (state) vs long-term (store)
- Multi-session, multi-user support in LangGraph ← key for production
- Streaming in LangGraph
- LangGraph persistence — checkpointing with SQLite/PostgreSQL

**Block 10 — Evaluation & Production** *(Tutorial section 16 + extras)*

- LangSmith — tracing, debugging, prompt versioning
- Evaluating LLM outputs — criteria-based, pairwise, reference-based

- RAGAs evaluation suite
- Prompt versioning and management
- Guardrails — input/output validation
- Cost tracking and token optimization

**Block 11 — Beyond LangChain (Add-ons)**

- Fine-tuning basics — when RAG is not enough
- LoRA / QLoRA — fine-tune open source models (LLaMA, Mistral)
- Ollama — running local LLMs, great for privacy-sensitive projects
- Function calling / Tool use directly with OpenAI API (without LangChain)
- Structured generation — Instructor library, Pydantic AI

**Practice Sites & Resources for LLM:**

- **LangChain Open Tutorial** — your main source, go chapter by chapter
- **LangGraph official docs** — langgraph.dev — for production agent patterns
- **Hugging Face Hub** — load, push models, read model cards
- **LangSmith** — free tier available, use it from day 1 to trace your chains
- **RAGAs GitHub** — for evaluation, understand the metrics
- **Simon Willison's blog (simonwillison.net)** — best practical LLM engineering blog, read weekly

📦 **Capstone Project 5:**

**"Production-Grade Multi-Agent AI Assistant"** — Build an agent that can: answer questions from your own PDF documents (RAG), search the web for real-time info, generate structured reports, maintain conversation memory across sessions, and has a simple chat UI (Streamlit or Gradio). Deploy on a free platform (Hugging Face Spaces or Render). This is your killer portfolio piece — one project that shows everything. 🔗 Use: LangGraph + LangSmith + FAISS + OpenAI API + Streamlit

---

☁️ **DOMAIN 6 — Cloud, MLOps & Deployment**

This is the difference between a DS who builds notebooks and one who builds products.

**Docker — Learn First**

- What containers are, why they matter

- Dockerfile — FROM, RUN, COPY, CMD, EXPOSE
- docker build, docker run, docker ps, docker logs
- docker-compose for multi-container apps (app + database)
- Pushing images to Docker Hub / AWS ECR

**FastAPI — Serve Every Model**

- Path operations, request/response models
- Pydantic models for validation
- Background tasks, async endpoints
- Swagger UI — auto docs
- Containerizing a FastAPI app

**AWS — Core Services Only**

- IAM — users, roles, policies
- S3 — store datasets, models, artifacts
- EC2 — spin up an instance, SSH in, run your app
- Lambda — serverless functions for lightweight inference
- SageMaker basics — deploy a model endpoint
- ECR — push Docker images
- **Target cert: AWS Cloud Practitioner** — ₹7,000 exam, recognized everywhere

**MLOps Fundamentals**

- MLflow — experiment tracking, model registry, artifact logging
- Git for ML — branching, tagging model versions
- DVC (Data Version Control) — versioning datasets
- GitHub Actions — CI/CD pipeline for ML (auto test + deploy on push)
- Model monitoring — Evidently AI (detect data drift, model degradation)
- Feature stores concept — what they are and why companies use them

**Practice Sites:**

- **AWS Free Tier** — create account, everything you need is free for 12 months
- **Whizlabs / Tutorials Dojo** — for AWS Cloud Practitioner exam prep (mock tests)

- **MLflow official docs** — mlflow.org, great examples
- **Evidently AI docs** — evidentlyai.com/docs
- **TestDriven.io** — best FastAPI + Docker tutorials available

📦 **Capstone Project 6:**

**"ML Model as a Deployed API"** — Take your churn prediction model from Domain 3. Wrap it in FastAPI. Dockerize it. Track experiments with MLflow. Deploy on AWS EC2. Set up a basic GitHub Actions pipeline so pushing code auto-deploys. Monitor with Evidently. This is a complete MLOps project. Most DS candidates at 3+ years cannot do this — you will stand apart completely.

---

🗃️ **DOMAIN 7 — SQL & Data Engineering Basics**

Underrated. Directly tested in 80% of DS interviews. Do not skip.

**SQL — Go Deep**

- SELECT, WHERE, GROUP BY, HAVING, ORDER BY
- JOINs — INNER, LEFT, RIGHT, FULL, CROSS, SELF
- Subqueries — correlated vs non-correlated
- Window functions — ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER, AVG OVER
- CTEs — WITH clause, recursive CTEs
- CASE WHEN, COALESCE, NULLIF
- Indexes — when they help, when they hurt
- Query optimization — EXPLAIN ANALYZE, execution plans
- Date/time functions — DATEDIFF, DATE_TRUNC, EXTRACT

**Data Engineering Basics (Awareness Level)**

- ETL vs ELT — what they mean, when each is used
- Apache Spark basics — why it exists, what PySpark does (just awareness)
- Airflow basics — DAGs, scheduling, what a pipeline orchestrator does
- Data warehouses — what Snowflake, BigQuery, Redshift are (concept level)

**Practice Sites:**

- **StrataScratch** — real DS interview SQL questions from FAANG companies. Do 30 questions minimum.
- **LeetCode SQL** — Easy + Medium, do 20 questions
- **Mode Analytics SQL Tutorial** — free, browser-based, real datasets
- **SQLZoo** — interactive SQL exercises, good for window functions

📦 **Capstone Project 7:**

**"SQL Analytics Case Study"** — Take a multi-table e-commerce dataset. Write 15+ SQL queries answering business questions — cohort analysis, retention rate, revenue by segment, top products, customer lifetime value. Format as a clean GitHub repo with a README. Interviewers love this. 🔗 Dataset: **db-fiddle.com** or **Mode's public warehouse datasets**

---

📐 **DOMAIN 8 — System Design for ML (Year 2 Focus)**

This is what takes you from 15 LPA to 25 LPA. Study this in year 2.

**Core Concepts**

- How to design a recommendation system — collaborative filtering, content-based, hybrid
- How to design a fraud detection system — real-time vs batch, feature engineering at scale
- How to design a RAG system for millions of documents — chunking strategy, vector DB choice, caching
- How to design a real-time ML prediction API — latency, throughput, scaling
- Feature stores — why companies like Uber/Airbnb built them
- Online vs offline evaluation of ML models
- Model serving — single model vs ensemble, canary deployment, shadow mode
- Data pipelines — batch vs streaming, Kafka concept, Lambda architecture

**Main Resource:**

- 📖 **"Designing Machine Learning Systems" by Chip Huyen** — read this cover to cover in year 2. It's the most important book for senior DS roles.
- **ML System Design (GitHub: chiphuyen/machine-learning-systems-design)** — free companion resources

- **The Pragmatic Engineer Newsletter** — paid but worth it for system design awareness
-

### 🔀 THE HYBRID ORDER TO FOLLOW

**Stage 1 — Build the mental model first (2-3 weeks)**

Do Hands-On LLMs chapters 1, 2, 3 first — in this order:

- Ch 1 → Introduction to Language Models — what they are, how they generate text
- Ch 2 → Tokens and Embeddings — the single most important concept to deeply understand
- Ch 3 → Looking Inside Transformer LLMs — attention, how information flows

Why first? Because when you then do LangChain tutorial and work with embeddings, vector stores, chunking — you'll *understand why* each decision matters instead of just copy-pasting code. This is what separates a senior engineer from a junior one in interviews.

**Stage 2 — Start building in parallel (ongoing)**

Once you finish Ch 1-3 of Hands-On LLMs, start LangChain Tutorial from Block 1 and go sequentially. From here, alternate like this:

| LangChain Tutorial | Pair with Hands-On LLMs |
|---|---|
| Blocks 1-2 (Basics, Prompts, Memory) | Ch 6 — Prompt Engineering |
| Blocks 3-4 (Loaders, Embeddings, Vector Stores) | Ch 8 — Semantic Search & RAG |
| Block 5-6 (Retrieval, RAG) | Ch 10 — Creating Text Embedding Models |
| Block 7-8 (Chains, Agents) | Ch 7 — Advanced Text Generation |
| Block 9 (LangGraph) | Ch 11-12 — Fine-tuning (read while building agents) |

**Stage 3 — Fine-tuning chapter standalone (Year 2)**

Hands-On LLMs Ch 11 and 12 on fine-tuning — read these separately when you reach the fine-tuning phase of your roadmap. They're the best visual explanation of LoRA and BERT fine-tuning available anywhere for free.

## DSA — Honest Direct Answer

**Short answer: Partial DSA. Not full DSA like a software engineer.**

**Here's the breakdown:**

❌ What You DON'T Need

- Trees, graphs, dynamic programming — this is SWE territory, not DS
- Competitive programming style problems — LeetCode Hard — not for you
- Complex sorting algorithm implementations — NumPy handles all of this
- Low level memory management — Python handles it

✅ What You DO Need — "DS-Relevant DSA"

Data Structures you must understand conceptually:

- Arrays and Hash Maps — you use these daily without realising it through lists and dicts
- Queues and Stacks — agent memory buffers, LangGraph state uses these patterns
- Hash Tables — how Python dicts work under the hood, why lookup is O(1)
- Graphs — actually important for you because LangGraph is literally a graph. Nodes, edges, directed graphs, traversal — understanding this makes LangGraph click instantly.
- Trees — decision trees in ML, JSON/nested data structures

Complexity awareness — not mastery:

- Understand O(n), O($n^2$), O(log n) — just enough to know when your code will be slow
- Know why vectorized NumPy is faster than a Python loop — this is complexity thinking applied to DS

Algorithms that directly appear in DS/AI:

- Search algorithms — binary search appears in hyperparameter tuning concepts
- Sorting — know it exists, NumPy/Pandas handles it
- Hashing — how vector databases index embeddings, how RAG retrieval works under the hood

- Graph traversal — BFS/DFS — directly relevant for LangGraph agent flows

📊 DSA Effort Allocation for Your Goal

| Topic | Priority | Why |
|---|---|---|
| Graphs + traversal | 🔴 High | LangGraph is literally this |
| Hash maps, arrays | 🔴 High | Daily Python use |
| Trees | 🟡 Medium | Decision trees, nested data |
| Complexity basics | 🟡 Medium | Write efficient code |
| Dynamic programming | 🟢 Skip | Not needed |
| Advanced graph algorithms | 🟢 Skip | Not needed |
| Bit manipulation | 🟢 Skip | Not needed |

🧠 **THE MENTAL FRAMEWORK YOU NEED TO BUILD (how much to be proficient)**

When any dataset lands in front of you, your brain should automatically run this sequence:

**Step 1 — Understand the problem type**

- Am I predicting a number? → Regression
- Am I predicting a category? → Classification
- Am I finding groups? → Clustering
- Am I detecting something rare? → Anomaly Detection
- Am I ranking things? → Ranking problem

**Step 2 — Understand the data**

- How many rows, how many columns?
- What are the data types — numerical, categorical, text, datetime, image?
- What's missing — how much, which columns, is it random or pattern?
- What's the target variable — is it balanced or skewed?
- What's the relationship between features and target?

**Step 3 — Know which model fits the situation**

| Situation | Go-to models |
|---|---|
| Small data, tabular, interpretability needed | Logistic Regression, Decision Tree |
| Tabular, need best accuracy | XGBoost, LightGBM |
| Lots of categorical features | CatBoost |
| High dimensional, sparse data | SVM, Logistic Regression with regularization |
| Text data | TF-IDF + Logistic Reg, or fine-tuned BERT |
| Imbalanced classes | XGBoost with scale_pos_weight, SMOTE + RF |
| Need to explain predictions to business | Random Forest + SHAP |
| Real-time inference needed | LightGBM (fastest prediction) |

**Step 4 — Build, evaluate, iterate** Train → Check metrics → Understand errors → Fix → Repeat

This mental loop is what Level 2 looks like. Every senior DS runs this automatically.

---

📏 **HOW DEEP IN NUMPY, PANDAS, SKLEARN**

**NumPy — Medium depth is enough**

You do NOT need to master everything. Master only this:

- Array creation, reshaping, slicing — must be second nature
- Broadcasting — understand it visually once, use confidently
- Vectorized operations — never write a loop where NumPy can do it
- dot, matmul, transpose, linalg basics
- random seed, random generation

**Test:** If you can manipulate any array shape without Googling — you're done with NumPy. Takes 1 week.

**Pandas — Go deep here, this is daily bread**

This is the one library where depth directly equals salary. Master:

- Every type of selection — loc, iloc, boolean masks, .query()

- Every type of groupby — single, multi, custom agg functions
- Every type of merge/join — and knowing when to use which
- Handling missing data — fillna strategies, interpolation, dropping logic
- apply, map, transform — and knowing which is faster
- String operations — .str accessor
- Datetime operations — resample, rolling, shift, diff
- Method chaining — writing clean Pandas without temp variables
- Memory optimization — dtypes, category dtype for strings

**Test:** Give yourself the Titanic dataset or any Kaggle CSV. If you can answer 10 business questions from it in under 20 minutes with clean code — you're at Level 2. Takes 3-4 weeks of daily practice.

**Sklearn — Know the pattern deeply, not every algorithm**

Sklearn has one beautiful pattern — fit, transform, predict. Master this:

- The full Pipeline — ColumnTransformer + preprocessing + model in one object
- Every preprocessing step — scalers, encoders, imputers and when to use each
- Cross-validation — why it exists, how to do it right without data leakage
- GridSearchCV and RandomizedSearchCV — hyperparameter tuning properly
- Model evaluation — not just accuracy, full classification report, ROC curves
- Feature importance, SHAP integration
- Saving and loading models — joblib, pickle

**Test:** Can you take a raw messy CSV, build a full sklearn Pipeline end-to-end with preprocessing + model + cross-validation + evaluation in one clean notebook without Googling the syntax? That's Level 2. Takes 4-6 weeks.

---

### 📚 BEST RESOURCES TO BUILD THIS INTUITION

**For the "what model to use" intuition — this is the most important:**

- **"Approaching Almost Any Machine Learning Problem" by Abhishek Thakur** — free PDF on his GitHub. This single book gives you the mental framework for any dataset. Read this before anything else. Abhishek is a 4x Kaggle Grandmaster.

- **Kaggle Learn — Intermediate ML** — free, browser-based, teaches pipelines and cross-validation the right way
- **StatQuest YouTube** — watch his Decision Tree, Random Forest, XGBoost, and ROC playlist. Nobody explains the intuition better.

**For Pandas depth:**

- **"Python for Data Analysis" by Wes McKinney** — Wes literally built Pandas. Chapter by chapter, do the exercises. Free PDF available.
- **Kaggle Pandas micro-course** — best quick exercises
- **StrataScratch** — real company interview Pandas questions

**For building the data instinct:**

- **Kaggle Notebooks** — this is your gym. Every week pick one popular dataset, read the top 3 EDA notebooks, see how experienced people think, then write your own from scratch.
- **"Storytelling with Data" by Cole Nussbaumer Knaflic** — teaches you how to read data like a business person not just a coder. Changes how you see any dataset permanently.

## 🧠 DEEP NEURAL NETWORKS — How Deep You Need to Go

### The Mental Model You Need

When someone gives you a problem, your brain should instantly know — is this a neural network problem or is sklearn enough? The answer is:

- Tabular data, under 100k rows → sklearn first, DNN rarely needed
- Images, audio, video → DNN always
- Text sequences, language → Transformers
- Tabular but massive scale with complex interactions → DNN can help

### What to Master

**Foundations — must be second nature**

- Forward pass — how data flows layer by layer, what activations do
- Backpropagation — understand the concept, not the calculus derivation. Know that gradients flow backward and weights update. That's enough.
- Loss functions — MSE for regression, CrossEntropy for classification, why each

- Optimizers — Adam is your default. Know why it's better than vanilla SGD. Know what learning rate does practically.
- Batch size and epochs — what happens when batch too small, too large
- Overfitting signals — training loss drops, val loss rises. Solutions — Dropout, BatchNorm, Early Stopping, more data, regularization

**PyTorch — your framework, learn this not Keras**

- Tensor operations — creation, reshaping, moving to GPU
- nn.Module — how to define a model properly
- DataLoader and Dataset — how to feed data efficiently
- Training loop — the 5 lines that never change: forward → loss → zero_grad → backward → step
- Saving, loading models — torch.save, torch.load
- Moving to GPU — .to(device), why it matters

**Test for DNN Level 2:** Build a neural network from scratch in PyTorch for a tabular problem, a binary classification problem, train it, plot training curves, implement early stopping, save the model. No tutorial open. If you can do this — you're Level 2 on DNN fundamentals. Takes 3-4 weeks.

---

👁️ **COMPUTER VISION — How Deep You Need to Go**

**The Mental Model**

When someone gives you an image problem, your brain should run this:

- Classify the whole image → CNN classification (ResNet, EfficientNet pretrained)
- Detect objects in image → YOLO (you know this already)
- Segment pixel by pixel → SegFormer or U-Net
- Compare two images similarity → Siamese networks or embedding similarity
- Generate images → Diffusion models (awareness level only for now)

**What to Master**

**Core CV skills**

- OpenCV preprocessing — resize, normalize, augment, morphological ops, contour detection. This is your daily toolkit.

- Transfer Learning — loading a pretrained ResNet or EfficientNet, freezing layers, replacing the head, fine-tuning. This is the most practical skill in CV.
- Data augmentation — torchvision transforms, Albumentations library. Knowing how to augment properly changes model performance dramatically.
- Training a CV model end to end — dataset → dataloader → model → train loop → evaluation → save
- Evaluation metrics for CV — Accuracy, Precision, Recall for classification. mAP, IoU for detection.

**YOLOv8 — upgrade your existing skill** You've done basic YOLO. Now add:

- Custom dataset training — labeling with Roboflow, training on your own classes
- Inference optimization — running YOLO fast, exporting to ONNX
- Serving YOLO as a FastAPI endpoint — wrap model in an API, accept image, return predictions as JSON

**What you DON'T need to go deep on yet**

- Building CNNs from scratch — use pretrained, always
- GANs — interesting but low ROI for your goals right now
- 3D vision, medical imaging — niche, not needed unless your work demands it

**Test for CV Level 2:** Take any image classification dataset from Roboflow. Fine-tune EfficientNet on it using transfer learning in PyTorch. Wrap the trained model in a FastAPI endpoint. Send an image via Postman, get a prediction back. If you can do this without tutorials open — you're Level 2. Takes 4-5 weeks.

---

### 🦜 GENAI & LLMs — How Deep You Need to Go

**The Mental Model**

When someone gives you a language or text problem, your brain should run this:

- Answer questions from documents → RAG pipeline
- Automate a multi-step workflow → Agent with tools
- Classify or extract from text → Fine-tuned BERT or prompt-based classification
- Summarize, rewrite, generate → Prompt engineering first, fine-tune only if prompts fail
- Need a chatbot with memory → LangGraph with persistence

- Company wants private LLM → Ollama + open source model

**What to Master**

**Prompt Engineering — deeper than most people go**

- Zero-shot, few-shot, chain-of-thought prompting
- System prompt design — how to constrain model behavior reliably
- Structured output prompting — making LLM return JSON consistently
- Prompt chaining — breaking complex tasks into smaller prompt steps
- Handling hallucinations — verification prompts, grounding techniques

**RAG — your current strength, go production-depth**

- Chunking strategies and their impact on retrieval quality — this is underrated
- Hybrid search — BM25 keyword + vector semantic search combined
- Reranking — why similarity score alone is not enough
- Advanced RAG patterns — HyDE, Self-RAG, Corrective RAG
- RAG evaluation — RAGAs metrics — faithfulness, answer relevancy, context precision
- Production RAG — caching, latency optimization, cost per query tracking

**LangGraph — this is your main differentiator, go deep**

- State management — defining what the agent remembers across steps
- Conditional routing — agent decides what to do next based on output
- Tool calling — building custom tools, connecting to real APIs and databases
- Supervisor + worker multi-agent pattern — orchestrator delegates to specialized agents
- Human in the loop — agent pauses, asks human, resumes
- Memory — short term in state, long term in vector store or database
- Multi-user, multi-session support — separate memory per user per session
- Streaming — real-time token streaming to frontend
- Persistence — checkpoint graph state so conversations survive server restarts

**Fine-tuning — awareness + basic hands-on**

- When to fine-tune vs when RAG is enough — very important decision

- LoRA and QLoRA — the efficient fine-tuning methods used in industry
- Fine-tune a small open source model (Mistral 7B or LLaMA 3) on Google Colab using QLoRA
- Push to Hugging Face Hub
- You don't need to be an expert here. One end-to-end fine-tuning done = interview talking point.

**LLM Ops — the production layer**

- LangSmith — trace every chain, debug failures, compare prompt versions
- Token cost tracking — know what each call costs, how to optimize
- Guardrails — input and output validation, blocking harmful or off-topic inputs
- Latency optimization — streaming, async calls, parallel chains

**Test for GenAI Level 2:** Build a multi-agent system where Agent 1 retrieves from a PDF knowledge base, Agent 2 searches the web, a Supervisor routes between them based on the question, responses stream in real-time, and conversation memory persists across sessions. Deploy it with a Streamlit UI. If you can build this — you are Level 2 and that project alone can get you a 15-18 LPA interview call. Takes 6-8 weeks.

---

### 🤖 AGENTIC AI — How Deep You Need to Go

**The Mental Model — this one is crucial**

Most people think agents are just "LLM that calls tools." That's Level 1 thinking. Level 2 thinking is:

An agent is a **stateful system that makes decisions over time.** It has memory, it has goals, it has tools, and it has error handling. The hard problems in agents are not the LLM call — they are state management, reliability, error recovery, and knowing when to stop.

**What to Master**

**Core agentic patterns — know all of these**

- ReAct — Reasoning + Acting loop. The base pattern everything builds on.
- Plan and Execute — agent makes a full plan first, then executes step by step
- Reflection — agent reviews its own output and improves it
- Supervisor / Orchestrator — one agent manages many specialist agents
- Parallelization — multiple agents run simultaneously, results merged

**Tool design — underrated skill**

- Writing good tool descriptions — the LLM reads your docstring to decide when to use a tool. Bad description = agent uses wrong tool.
- Tool error handling — what happens when a tool fails, how agent recovers
- Tool types — database query tools, API call tools, file tools, code execution tools, search tools

**Memory architecture for production agents**

- Conversation memory — what the user said this session
- Entity memory — facts extracted and stored about users, topics
- Episodic memory — past conversation summaries stored in vector DB
- Procedural memory — agent remembers how to do tasks it's done before

**Multi-agent coordination**

- When to use one agent vs multiple agents
- How agents communicate — shared state vs message passing
- Conflict resolution when agents disagree
- Cost and latency tradeoffs — more agents = more LLM calls = more cost

**Test for Agentic Level 2:** Build a research agent that takes a business question, plans its research steps, uses a web search tool and a document retrieval tool, reflects on whether it has enough information, writes a structured report, and knows when it's done vs when it needs more data. This is what companies are literally trying to hire for right now. Takes 4-6 weeks after LangGraph fundamentals.

---

⏱️ **TIME TO LEVEL 2 — FULL PICTURE**

| Domain | Time Given Your Background |
|---|---|
| DNN Fundamentals + PyTorch | 3-4 weeks |
| Computer Vision Level 2 | 4-5 weeks |
| GenAI + RAG production depth | 6-8 weeks |
| LangGraph + Agentic patterns | 4-6 weeks |
| Fine-tuning basics | 2 weeks |

**Total: 5-6 months** running GenAI track parallel to basics. This fits perfectly inside your 7-8 month plan.

---

### 📚 BEST RESOURCES FOR THESE

**DNN + PyTorch:**

- fast.ai — practical deep learning course, project first, free, the best
- PyTorch official tutorials — pytorch.org, just do them sequentially

**Computer Vision:**

- Roboflow blog — best practical CV content anywhere
- Ultralytics YOLOv8 docs — for advancing your YOLO skills

**GenAI + LangGraph:**

- LangChain Open Tutorial — your already chosen source, perfect
- LangGraph official docs — langgraph.dev — read this alongside the tutorial
- LangSmith docs — for evaluation and tracing

**Agentic patterns:**

- Andrew Ng's Agentic AI short courses on DeepLearning.AI — all free, each is 1-2 hours, covers ReAct, multi-agent, planning
- LangGraph How-To guides — real production patterns with code

**Fine-tuning:**

- Maxime Labonne's LLM course on GitHub — completely free, covers QLoRA fine-tuning end to end, best resource for this